

```

1  # Note 13: Cross - Validation Techniques
2  # =====
3
4  # Import Libraries
5  from sklearn.model_selection import train_test_split
6  from sklearn.linear_model import LogisticRegression
7  from sklearn.svm import SVC
8  from sklearn.ensemble import RandomForestClassifier
9  from sklearn.model_selection import KFold
10 from sklearn.datasets import load_digits
11 from sklearn.model_selection import StratifiedKFold
12 from sklearn.model_selection import cross_val_score
13
14 # Load dataset into variable
15 digits = load_digits()
16
17 # Split dataset into training and testing sets (70% training, 30% testing)
18 X_train, X_test, y_train, y_test = train_test_split(digits.data, digits.target,
19 test_size=0.3)
20
21 # Create Logistic Regression model
22 lr = LogisticRegression(max_iter=500)
23
24 # Train Logistic Regression model
25 lr.fit(X_train, y_train)
26
27 # Evaluate Logistic Regression model accuracy
28 print(lr.score(X_test, y_test))
29
30 # Create Support Vector Machine model
31 svm = SVC()
32
33 # Train SVM model
34 svm.fit(X_train, y_train)
35
36 # Evaluate SVM model accuracy
37 print(svm.score(X_test, y_test))
38
39 # Create Random Forest model with 40 trees
40 rf = RandomForestClassifier(n_estimators=40)
41
42 # Train Random Forest model
43 rf.fit(X_train, y_train)
44
45 # Evaluate Random Forest model accuracy
46 print(rf.score(X_test, y_test))
47
48 # Create K-Fold cross-validator with 3 splits
49 kf = KFold(n_splits=3)
50
51 # Print K-Fold configuration
52 print(kf)
53
54 # Show how indices are split in K-Fold
55 for train_index, test_index in kf.split([1, 2, 3, 4, 5, 6, 7, 8, 9]): print(
56 train_index, test_index)
57
58 # Function to train a model and return its accuracy
59 def get_score(model, X_train, X_test, y_train, y_test):
60     model.fit(X_train, y_train)
61     return model.score(X_test, y_test)
62
63 # Evaluate Logistic Regression using helper function
64 print(get_score(LogisticRegression(max_iter=500), X_train, X_test, y_train, y_test))
65
66 # Evaluate SVM using helper function
67 print(get_score(SVC(), X_train, X_test, y_train, y_test))
68
69 # Create Stratified K-Fold cross-validator with 3 splits
70 folds = StratifiedKFold(n_splits=3)
71
72 # Lists to store model scores for each fold

```

```

71 scores_l = []
72 scores_SVM = []
73 scores_rf = []
74
75 # Perform K-Fold cross-validation manually
76 for train_index, test_index in kf.split(digits.data):
77     X_train, X_test, y_train, y_test = digits.data[train_index], digits.data[
78         test_index], digits.target[train_index], digits.target[test_index]
79     scores_l.append(get_score(LogisticRegression(max_iter=500), X_train, X_test,
80         y_train, y_test))
81     scores_SVM.append(get_score(SVC(), X_train, X_test, y_train, y_test))
82     scores_rf.append(get_score(RandomForestClassifier(n_estimators=40), X_train,
83         X_test, y_train, y_test))
84
85 # Print results of manual cross-validation for each model
86 print("LR : ", scores_l)
87 print("SVM : ", scores_SVM)
88 print("RF : ", scores_rf)
89
90 # Perform cross-validation using built-in function (Logistic Regression)
91 print(cross_val_score(LogisticRegression(max_iter=500), digits.data, digits.target))
92
93 # Perform cross-validation using built-in function (SVM)
94 print(cross_val_score(SVC(), digits.data, digits.target))
95
96 # Parameter tuning example (changing number of trees in Random Forest)
97 print(cross_val_score(RandomForestClassifier(n_estimators=40), digits.data, digits.
98     target))
99
100 # Import Libraries
101 import numpy as np
102 import pandas as pd
103 from sklearn.model_selection import train_test_split, cross_val_score
104 from sklearn.neighbors import KNeighborsClassifier
105 from sklearn.metrics import accuracy_score
106 from sklearn.naive_bayes import GaussianNB
107 from sklearn.svm import SVC
108
109 # 01. Dataset Preparation
110 iris = pd.read_csv("Iris.csv")
111 X = iris.drop(columns=["Species", "Id"])
112 y = iris["Species"]
113
114 # 02. Split dataset
115 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
116
117 # 03. Create KNN model
118 model1 = KNeighborsClassifier(n_neighbors=3)
119
120 # 04. Train model
121 model1.fit(X_train, y_train)
122
123 # 05. Predict
124 y_pred = model1.predict(X_test)
125
126 # 06. Accuracy Score
127 print(f"Accuracy as a Percentage for KNN Model : {accuracy_score(y_test, y_pred) * 100
128     :.2f}%")
129
130 # 07. Create Naive Bayes model
131 model2 = GaussianNB()
132
133 # 08. Train model
134 model2.fit(X_train, y_train)
135
136 # 09. Predict
137 y_pred = model2.predict(X_test)
138
139 # 10. Accuracy Score

```

```

138 print(f"Accuracy as a Percentage for Naive Bayes Model : {accuracy_score(y_test,
139 y_pred) * 100:.2f}%")
140
141 # 11. Cross Validation
142 scores = cross_val_score(model1, X, y, cv=5)
143 print("Cross Validation Mean For KNN Model : ", (np.mean(scores) * 100))
144
145 # 12. SVM Model
146 model3 = SVC(kernel="rbf", gamma="auto")
147 model3.fit(X_train, y_train)
148 y_pred = model3.predict(X_test)
149 print(f"Accuracy as a Percentage for Naive Bayes Model : {accuracy_score(y_test,
150 y_pred) * 100:.2f}%")
151 scores = cross_val_score(model3, X, y, cv=5)
152 print("Cross Validation Mean For SVM : ", (np.mean(scores) * 100))
153
154 # =====
155 # Summary of Cross - Val Score
156 # =====
157
158 # from sklearn.model_selection import cross_val_score
159
160 # model = YourModel()
161 # scores = cross_val_score(
162 #     model,
163 #     X,
164 #     y,
165 #     cv=5
166 # )
167 # print("CV Scores:", scores)
168 # print("Mean CV Score:", scores.mean())
169
170 # For Regression
171 # =====
172
173 # scores = cross_val_score(
174 #     model,
175 #     X,
176 #     y,
177 #     cv=5,
178 #     scoring="r2"
179 # )

```